

Keywords: diffusion LLMs • KV caching • speculative decoding • inference acceleration

Flash-dLLM

IO-Aware KV Caching and Parallel Decoding for Fast, Memory-Efficient Diffusion LLMs

By Quan Nguyen-Tri, Mukul Ranjan & Zhiqiang Shen (VILA Lab, MBZUAI)

arXiv:2609.26796 • September 2026

An IO-aware fused KV-cache kernel combined with a self-drafting decode strategy eliminates the HBM bandwidth bottleneck that has crippled diffusion LLM inference, delivering $5.1\times$ and $11.0\times$ speedups over the strongest prior method on GSM8K and HumanEval without any model modification.

AT A GLANCE

Problem: Diffusion LLMs require full KV-cache reads at every denoising step; GPU HBM I/O is the dominant bottleneck, not compute.

Why it matters: dLLMs promise parallel token generation, but the IO bottleneck has prevented them from reaching autoregressive LLM speeds.

Method: Fused IO-aware CUDA kernel reduces redundant HBM reads; the dLLM itself serves as drafter and verifier, eliminating auxiliary models.

Result: $5.1\times$ speedup on GSM8K and $11.0\times$ on HumanEval vs. Elastic-Cache; improved memory efficiency at equal quality.

Evidence: Mathematical reasoning and code generation benchmarks on A10G, A100, and L4 GPUs.

The Big Picture

Diffusion language models (dLLMs) generate text by iteratively denoising masked tokens rather than decoding one token at a time. Their non-autoregressive structure should, in principle, allow far more parallelism than autoregressive models. In practice, dLLM inference has remained slow because every denoising step requires reading the entire KV cache—a cost proportional to sequence length that must be paid at each of the T iterations.

Flash-dLLM diagnoses this as an I/O bottleneck: on all tested GPUs, the attention computation in a KV-cache-enabled dLLM is *memory-bandwidth-bound*, not compute-bound. Arithmetic intensity (FLOPs per HBM byte) falls far below the GPU’s compute-to-memory ratio, meaning that the GPU arithmetic units are idle most of the time, waiting for data.

Why Existing Approaches Fall Short

Standard autoregressive KV caching exploits left-to-right structure; dLLMs access the full bidirectional context at every step, so per-step IO cost is proportional to sequence length regardless of generation position.

Elastic-Cache and token eviction reduce cache size but preserve the naive kernel structure: the remaining tokens still require an unfused read-attend-write pipeline that is IO-inefficient.

Auxiliary-model draft-and-verify (as used in speculative decoding for autoregressive models) adds a second model to load and run, increasing memory and latency per step.

Flash Attention for autoregressive models was designed for compute-bound settings; its tiling strategy does not generalise to the full-sequence, bidirectional reads required by dLLMs.

Core Mechanism

IO-Aware Fused Kernel. Flash-dLLM fuses the KV-cache read, attention computation, and output write into a single kernel. Intermediate values are kept in on-chip SRAM, eliminating round-trips to HBM for the intermediate attention maps. The tiling strategy is tuned for the bidirectional, full-sequence access pattern of dLLMs rather than the causal sliding-window pattern of autoregressive models.

Self-Drafting Decode Strategy. Early denoising steps operate at high noise levels and produce coarse but fast predictions. Flash-dLLM uses these as a draft: the KV cache computed in early steps is reused by later steps, and positions where the early draft matches the final refined output avoid recomputation. The dLLM itself is both drafter and verifier—no auxiliary model is loaded.

Technical Formulation

Let $\mathbf{x}^{(t)}$ be the partially denoised sequence at step t , f_θ the dLLM, L the sequence length, d the head dimension, H the number of heads, l the number of layers.

Naive KV-cache IO cost per step:

$$\text{IO}(t) = 2 \cdot L \cdot d \cdot H \cdot l \cdot \text{sizeof}(\text{fp16})$$

Fused-kernel IO cost (tile size B_{SRAM}):

$$\text{IO}_{\text{fused}}(t) = \frac{L \cdot d \cdot H \cdot l}{B_{\text{SRAM}}} \cdot \text{sizeof}(\text{fp16})$$

The draft-and-verify step amortises IO over T_{draft} steps: positions where the draft $\hat{\mathbf{x}}^{(0)}$ at step t matches the verified output at step $t-1$ skip re-attention, contributing a further multiplicative reduction in total IO.

Learning or Inference Procedure

Flash-dLLM requires no training and no model modification:

1. Compile the IO-aware fused CUDA kernel for the target GPU architecture.
2. Run the first T_{draft} denoising steps with the fused kernel; record draft token predictions from early steps.
3. Verify drafts in the remaining $T - T_{\text{draft}}$ steps; reuse KV cache for positions where draft is accepted.
4. Return the fully denoised sequence.

The kernel and draft parameters are tuned once per GPU model; no per-task adaptation is needed.

What the Guarantee Says

The fused kernel is numerically exact: it computes the same attention values as the unfused baseline up to floating-point rounding. Speedup is achieved purely by reducing memory traffic, not by approximation. The self-drafting strategy introduces an optimism bias (assuming later steps will agree with earlier ones), but the verification step guarantees that accepted tokens are correct.

Experimental Findings

- Versus Elastic-Cache (strongest prior baseline): **5.1 $\times$ speedup** on GSM8K (mathematical reasoning); **11.0 $\times$ speedup** on HumanEval (code generation)
- Memory efficiency: lower peak HBM usage at equal or better quality
- Code benchmark shows larger gains because longer sequences and sparser denoising trajectories amplify IO savings
- Inference speed approaches autoregressive LLMs with Flash Attention at the same model size

Ablations and Interpretation

Fused kernel alone accounts for 3–5 $\times$ of the speedup; the self-drafting strategy contributes 1.5–2 $\times$.

Draft length T_{draft} : Shorter drafts reduce reuse and speedup; longer drafts risk more corrections but still improve throughput because IO savings outweigh correction costs.

IO profile: On all tested GPUs, attention accounts for over 70% of total inference time with KV caching, confirming the IO-bottleneck hypothesis.

Arithmetic intensity: Before the fused kernel, dLLM attention falls below 10% of the GPU roofline; after, it approaches 60–80%, indicating near-full utilisation of HBM bandwidth.

Reference

Quan Nguyen-Tri, Mukul Ranjan, Zhiqiang Shen. **Flash-dLLM: IO-Aware KV Caching and Parallel Decoding for Fast, Memory-Efficient Diffusion LLMs**. arXiv:2609.26796, September 2026.

<https://arxiv.org/abs/2609.26796>